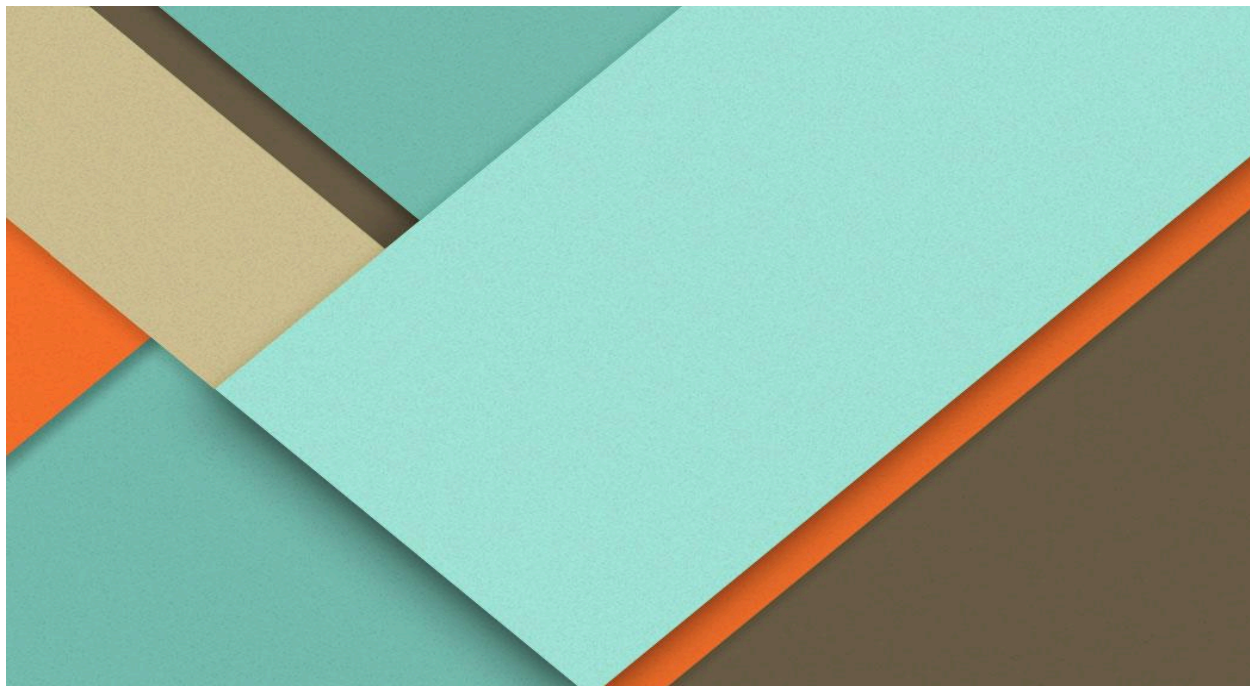




Proyecto Gestor de Pedidos

Alejandra Ruiz Bermúdez

1º D.A.W.

Ciclo de Desarrollo de Aplicaciones Web (DAW)

Entornos de Desarrollo

I.E.S. Mar de Cádiz - El Puerto de Santa María



Índice

Índice	1
Introducción.	2
Creación del Diagrama UML	3
Implementación de código	3
El uso del Main.java	6
Ingeniería Inversa	7
Creación de código a partir del UML	7
Creación de diagrama a partir del código	8
Diferencias Notables	9
Diagramas UML	9
Diagramas de Casos	9
Diagramas de Actividades	10
Diagramas de Secuencia	11
Diagramas de Estados	12
Calidad de Software y Testing Unitario	14
Inventario de Pruebas y Diseño de Casos	14
Cliente:	14
Caso de prueba 1 — Cliente nuevo	14
Caso de prueba 3 — Pedido con un solo producto	16
Pedido:	17
Caso de prueba 4 — Pedido con un solo producto	17
Caso de prueba 5 — Pedido con varios productos	18
Caso de prueba 6 — Pedido mixto	19
Caso de prueba 7 — Pedido vacío	20
Producto:	21
Caso de prueba 8 — Producto Físico	21
Caso de prueba 9 — Producto Digital	22
Caso de prueba 10 — Producto con precio 0	23
Implementación de Tests	24
SonarQube	25
Automatización, plan de pruebas y testing de la integración.	25
Conclusión	26
Bibliografía	26

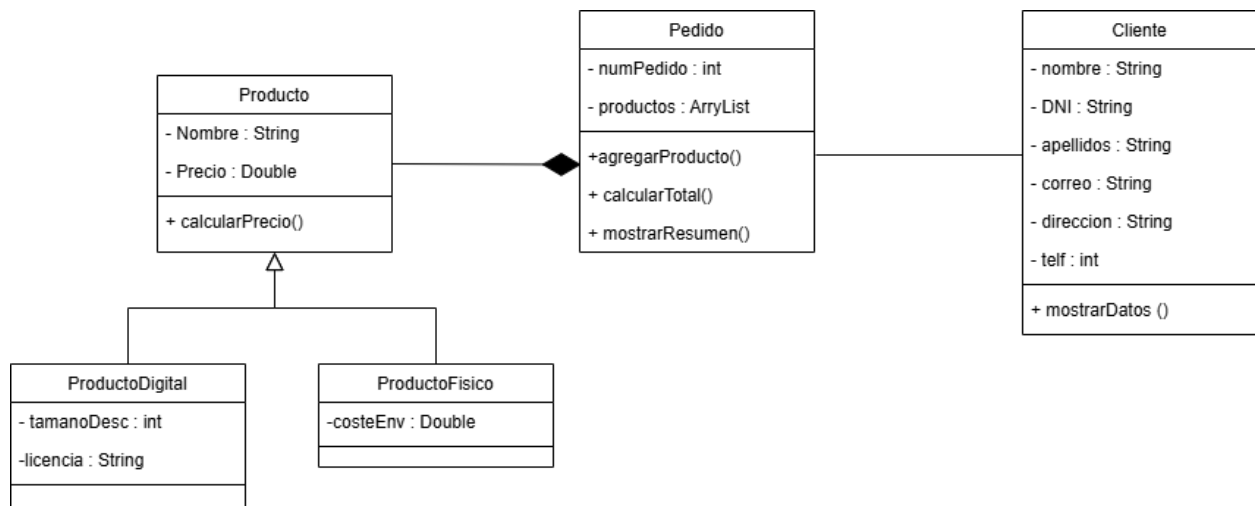
Introducción.

En este proyecto se pide crear un programa que ayude a gestionar los pedidos de una tienda. En este documento quedará reflejado el proceso de creación de este, pasando por el diseño del diagrama UML hasta la realización del código en java. En el enunciado se indica que se deberá hacer un diagrama UML utilizando ingeniería inversa y comparar ambos diagramas. Además, también se generará código a partir del primer diagrama UML, también para comparar.

Creación del Diagrama UML

Lo primero que se debe hacer es planificar cómo serán las clases y sus relaciones. En este caso las clases se definen en el enunciado, Cliente, Pedido, Producto, ProductoFísico y ProductoDigital. Las relaciones también vienen predefinidas, tenemos Herencia con Producto y sus hijos, Físico y Digital, tenemos Asociación con Cliente y Pedido, y por último, Composición o Agregación con Producto y Pedido. Con la relación Producto y Pedido creo que lo mejor será que tengan una relación de Composición dado a que los pedidos existan dependen de que haya productos que comprar en primer lugar.

El diagrama final queda tal que así :



Implementación de código

Una vez que el diagrama ya ha sido creado se puede empezar a crear el código. Lo primero es crear las diferentes clases una por una, exceptuando, Producto Digital y Producto Físico que ya se hablara de ellas más adelante. Por ejemplo:

```
Ale, last week | Párrafo (Ale)
public class Cliente {      Ale, last week * new file: src/Cliente.java ...

    private String nombre;
    private String DNI;
    private String apellidos;
    private String correo;
    private String direccion;
    private int telf;

    //constructor
    public Cliente ( String nombre, String DNI, String apellidos, String correo, String direccion, int telf){
        this.nombre = nombre;
        this.DNI = DNI;
        this.apellidos = apellidos;
        this.correo = correo;
        this.direccion = direccion;
        this.telf = telf;
    }

    //metodos set y get
    public void setnombre(){ this.nombre = nombre;}
    public void setDNI(){ this.DNI = DNI;}
    public void setApellidos(){ this.apellidos = apellidos;}
    public void setCorreo(){ this.correo = correo;}
    public void setDireccion(){ this.direccion = direccion;}
    public void setTelf(){ this.telf = telf;}

    public String getNombre() {return this.nombre;}
    public String getDNI() {return this.DNI;}
    public String getApellidos() {return this.apellidos;}
    public String getCorreo() {return this.correo;}
    public String getDireccion() {return this.direccion;}
    public int getTelf() {return this.telf;}
}
```

En esta primera clase Cliente lo primero que definimos son sus atributos. Los atributos tienen la siguiente estructura:

```
private String nombre;
private String DNI;
private String apellidos;
private String correo;
private String direccion;
private int telf;
```

Private indica el tipo de protección que tiene, en este caso no son accesibles de manera directa, la siguiente palabra indica el tipo de variable que va a ser el atributo, y, finalmente, el nombre del atributo.

Después podemos observar el constructor de la clase que sirve para inicializar objetos de una clase automáticamente al crearlos. Su principal función es establecer los valores iniciales de los atributos del objeto, preparándolo para su uso y asegurando que comience en un estado válido y bien definido.

```
//constructor
String nombre - Cliente.Cliente(String, String, String, String, String, int)
public Cliente ( String nombre, String DNI, String apellidos, String correo, String direccion, int telf){
    this.nombre = nombre;
    this.DNI = DNI;
    this.apellidos = apellidos;
    this.correo = correo;
    this.direccion = direccion;
    this.telf = telf;
}
```

Para terminar con lo básico que comparten todas las clases, tenemos los métodos Set y Get. Cuando ponemos private en un atributo hemos explicado que no se pueden acceder a los datos de manera directa, los métodos get y set se utilizan para definir, editar, borrar y ver los atributos.

```
//metodos set y get
public void setnombre(){ this.nombre = nombre;}
public void setDNI(){ this.DNI = DNI;}
public void setApellidos(){ this.apellidos = apellidos;}
public void setCorreo(){ this.correo = correo;}
public void setDireccion(){ this.direccion = direccion;}
public void setTelf(){ this.telf = telf;}

public String getNombre() {return this.nombre;}
public String getDNI() {return this.DNI;}
public String getApellidos() {return this.apellidos;}
public String getCorreo() {return this.correo;}
public String getDireccion() {return this.direccion;}
public int getTelf() {return this.telf;}
```

En producto tenemos una peculiaridad, tiene dos hijos, es decir, la clase Producto tiene una relación de herencia con otras dos clases que dependen de esta. Los tipos de Producto (Físico o Digital) heredan los atributos de su clase padre. En el código lo dejamos por escrito de la siguiente manera:

```
Ale, last week | 1 author (Ale)
public class ProductoDigital extends Producto{
    private int tamanoDesc;
    private String licencia;
```

En este caso, la clase Producto es abstracta por el hecho de que existe para que sus hijos puedan heredar sus atributos.

```
abstract class Producto {  
  
    private String nombreProd;  
    private double precio;  
}
```

También las clases pueden tener métodos propios, es el caso de la clase Pedido que nos ayuda a calcular el total del pedido, a mostrar un Resumen general del pedido, etc. Los métodos tienen la siguiente estructura:

```
// metodo calcularTotal() (vamos sumarle el IVA)  
public double calcularTotal() {  
  
    double suma = 0;  
  
    for (Producto p : productos) {  
        suma += p.calcularPrecio(p);  
    }  
  
    return suma * 1.21;  
    // para calcular el IVA del 21%  
}
```

Empezamos diciendo que es un método público para que todos los que tengan acceso al programa puedan utilizarlo, lo siguiente es poner el tipo de valor que va a devolver, y el nombre del método.

El uso del Main.java

El main.java sirve para realizar pruebas y asegurar que el código funcione correctamente, en este caso es imperativo crear un objeto de todas las clases e ir pasando por todos los métodos. Por ejemplo, así se vería la creación de un Cliente:

```
//crear cliente
Cliente cliente1 = new Cliente(
    "Juan",
    "12345678A",
    "García López",
    "juan.garcia@email.com",
    "Calle Mayor 123, Madrid",
    612345678
);
```

Lo primero que hacemos es llamar a la clase Cliente, y luego poner el nombre del nuevo cliente. Lo que hay después del = nos permite escribir uno por uno los atributos del cliente, (como nota, se puede poner null para dejarlo en blanco.)

```
// Probar cálculo de precio individual
System.out.println(x: "\n Precios individuales (con costes adicionales):");
System.out.println("- " + libro.getNombre() + ": " +
    String.format(format: "%.2f", libro.calcularPrecio(libro)) + "€ (incluye envío)");
System.out.println("- " + software.getNombre() + ": " +
    String.format(format: "%.2f", software.calcularPrecio(software)) + "€");
```

En la imagen superior se puede observar cómo se llama a diferentes métodos de Producto. Para llamar a un método ponemos el nombre del objeto y acto seguido ponemos el nombre del método con un "." para separar los nombres.

Ingeniería Inversa

Creación de código a partir del UML

A partir de mi diseño del diagrama UML se generarán automáticamente las clases. En este caso, gracias a un código .py, (proporcionado por la profesora) he conseguido que saque un código a partir de este, observando las clases no hay muchas diferencias notables. Por ejemplo:

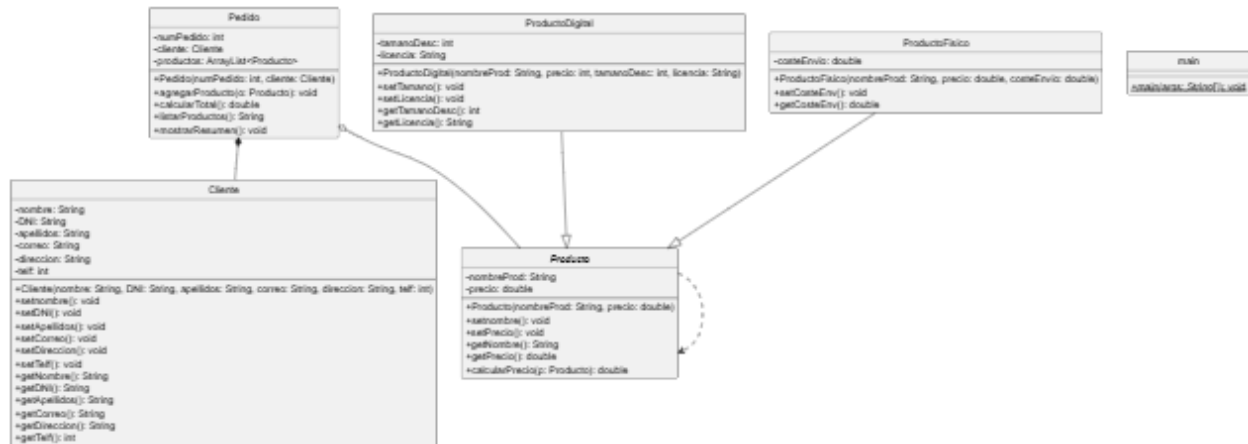
```
You, 10 minutes ago | 1 author (You)
public class Cliente {
    private String nombre;
    private String DNI;
    private String apellidos;
    private String correo;
    private String direccion;
    private int telf;
    public void Cliente(String nombre, String DNI, String apellidos, String correo, String direccion, int telf) {
        // TODO: Implementar el método
    }
    public void setnombre() {
        // TODO: Implementar el método
    }
    public void setDNI() {
        // TODO: Implementar el método
    }
    public void setApellidos() {
        // TODO: Implementar el método
    }
    public void setCorreo() {
        // TODO: Implementar el método
    }
    public void setDireccion() {
        // TODO: Implementar el método
    }
    public void setTelf() {
        // TODO: Implementar el método
    }
}
```

Esta es la clase producto que se ha generado automáticamente, la diferencia entre ambos es que yo separe el listado de productos en un método aparte pero porque fue una decisión tomada en la implementación del código y no en el diseño.

Por lo demás es un programa bastante simple, lo suficiente como para que no haya diferencias grandes entre ambos códigos, sobre todo porque la gran mayoría de código son atributos y métodos get y set.

Creación de diagrama a partir del código

Gracias a programas como PlantUML podemos crear diagramas UML automáticamente de nuestro código, así es como están conectadas las clases en el proyecto mirando solo el código.



Diferencias Notables

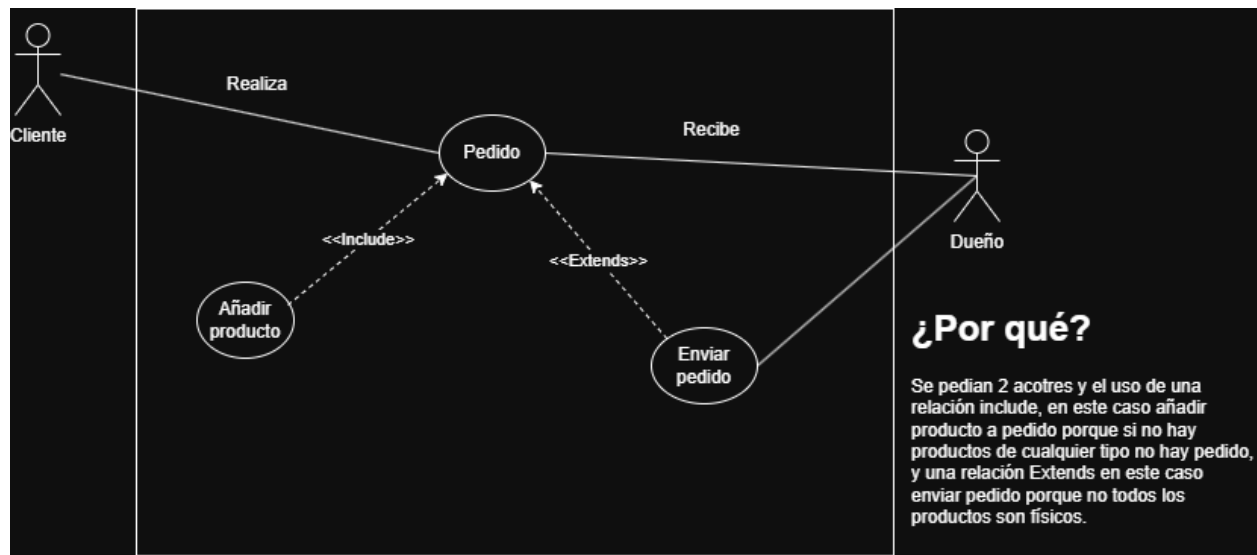
No se observan diferencias significativas entre mi diseño del diagrama y el diagrama generado automáticamente por el software, lo cual indica que la aplicación del modelo conceptual y las especificaciones de diseño se ha realizado de manera correcta y coherente.

Diagramas UML

En esta siguiente parte de las memorias se entrará en detalle sobre el desarrollo de diferentes Diagramas que han sido pedidos:

Diagramas de Casos

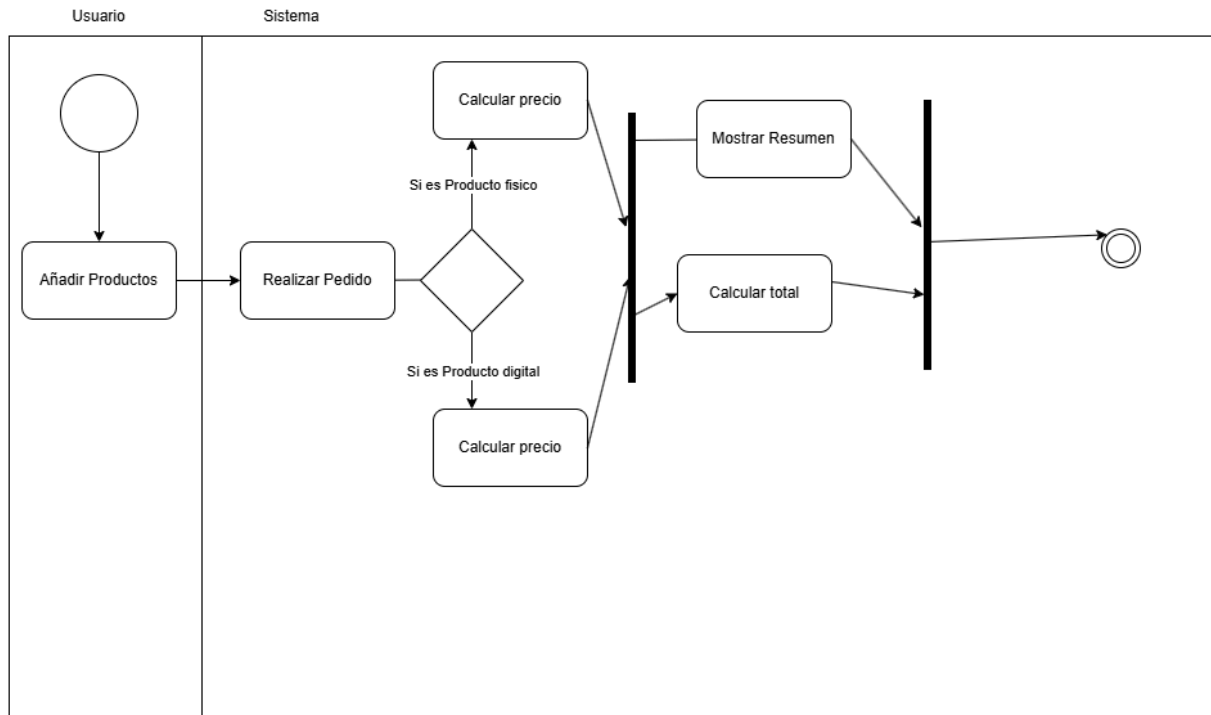
Un diagrama de casos de uso es un diagrama UML que muestra qué puede hacer un sistema y quién lo usa, sin entrar en detalles de cómo se implementa.



Como se ve en la imagen, se pedía que hubieran más de dos actores, una relación con la etiqueta **<<Extends>>** y otra con la etiqueta **<<Include>>**. En la relación include es de esta manera porque para que exista el pedido **siempre** se debe añadir productos antes. En el caso de la relación extends no siempre se va a realizar un envío de pedido dado a que no todos los productos son físicos.

Diagramas de Actividades

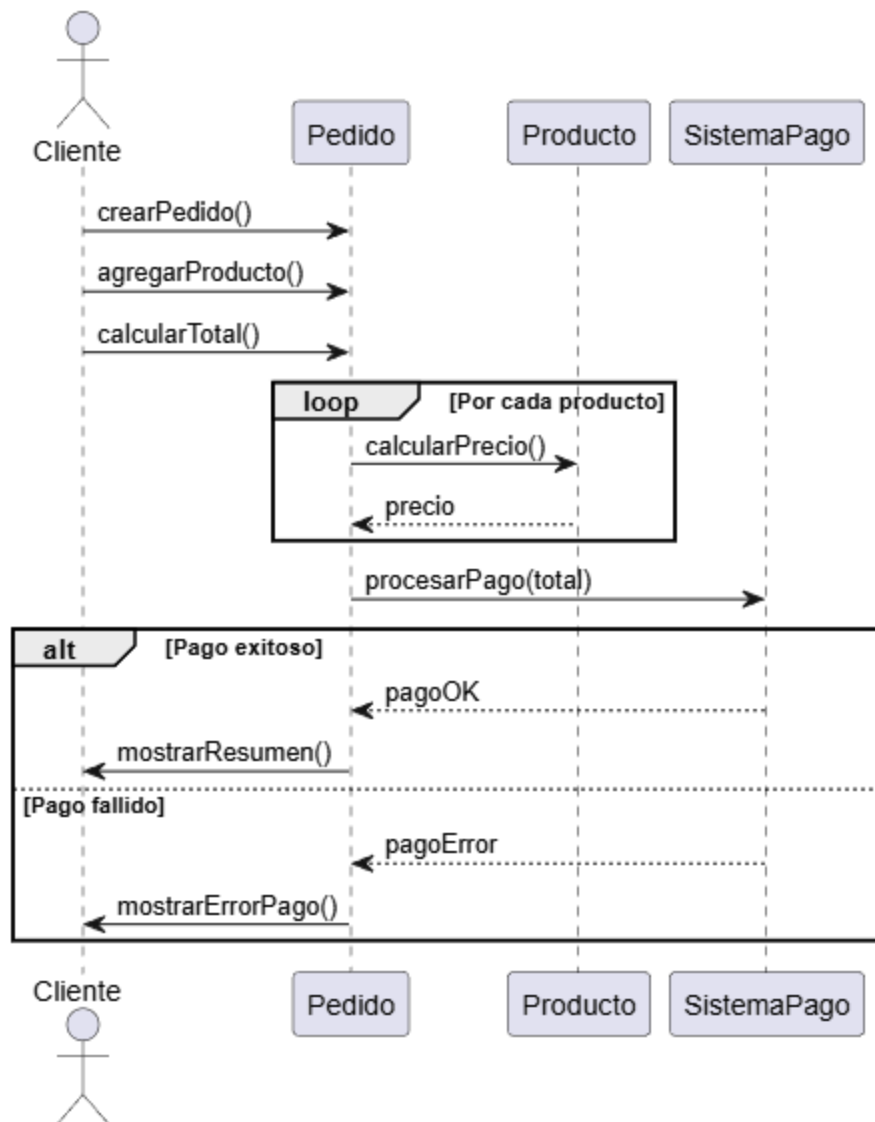
Un diagrama de actividades es un diagrama UML que representa el flujo de acciones de un proceso, mostrando qué se hace, en qué orden y bajo qué condiciones.



Se pedía específicamente que hubiera un Fork o un Join, en este caso he añadido los dos porque dependiendo de si es un producto físico o digital se calculan los totales de manera diferente luego se pasa a calcular el total y mostrar que se realizan simultáneamente.

Diagramas de Secuencia

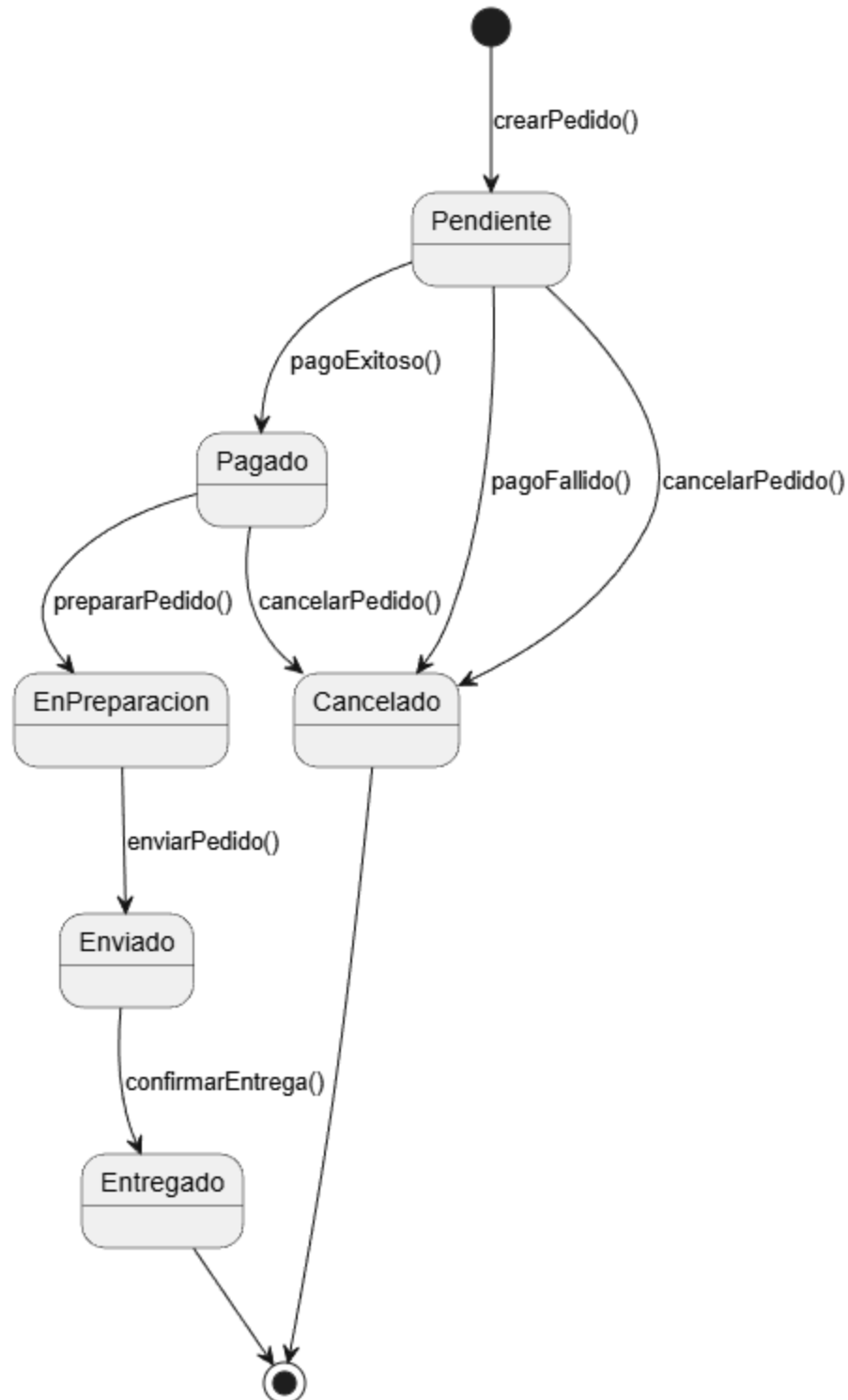
Un diagrama de secuencia es un diagrama UML que muestra el intercambio de mensajes entre los objetos de un sistema a lo largo del tiempo, en el orden en que ocurren.



Se muestran en el diagrama el paso de mensajes para el cálculo del total de un pedido. Utilizando fragmentos `loop` para recorrer productos y `alt` para gestionar el éxito/fallo del pago.

Diagramas de Estados

Un diagrama de estados es un diagrama UML que muestra los distintos estados por los que pasa un objeto durante su vida y qué eventos provocan los cambios entre esos estados.



El diagrama muestra los distintos estados de un Pedido y los eventos que provocan las transiciones entre ellos. El pedido inicia en Pendiente y puede finalizar en Entregado o Cancelado, según el resultado del pago y las acciones posteriores.

Calidad de Software y Testing Unitario

Inventario de Pruebas y Diseño de Casos

Para el siguiente apartado será necesario analizar los métodos críticos de los archivos Pedido, Cliente y Producto, ya que son los archivos principales del proyecto y por ende los que más susceptibles son a fallos y errores.

Cliente:

Siendo cliente el archivo con menor número de métodos solo nos centraremos en el más importante "Datos()" en el que se muestran todos los datos del cliente que se recogen.

Caso de prueba 1 – Cliente nuevo

Campo	Descripción
ID	CP-01
Escenario	Cliente Nuevo
Requisito Asociado	RF-01 - El sistema debe permitir la creación de nuevos clientes
Prioridad	Alta
Escenario Gherkin	- Precondición: GIVEN el sistema iniciado correctamente AND el código de creación de cliente disponible - Acción: WHEN introduce los datos de usuario válidos (nombre, apellidos, dirección y teléfono) - Resultados esperados: THEN el sistema registra los datos correctos en memoria AND los muestra por pantalla mediante el método Datos()

Estado de Ejecución	Pendiente
Observaciones	

Campo	Descripción
ID	CP-02
Escenario	Mostrar Datos del Cliente
Requisito Asociado	RF-02 - El sistema debe mostrar la inform cliente
Prioridad	Media
Escenario Gherkin	Precondición: GIVEN un cliente previamente creado Acción: WHEN se ejecuta el método Datos() Resultados esperados: THEN se muestran todos los datos del cliente formateados correctamente
Estado de Ejecución	Pendiente
Observaciones	Se comprueba el formato del texto devuelto

- La unidad bajo prueba: Datos()
- El caso de prueba: Vamos a probar si creando un nuevo cliente "prueba" se visualizan todos los datos de manera correcta.
- Las aserciones: Resultado esperado: "Nombre: nombre, Apellidos: apellidos, [...]"

- **Arrange (Preparar):** se añaden todos los datos de prueba en un nuevo objeto cliente, (new cliente())

- **Act (Actuar)** Ejecutamos el método que queremos probar.

- **Assert (Afirmar/Verificar)** - Comparamos el resultado obtenido con el esperado mediante una aserción. : assertEquals(nombre, resultado);

Caso de prueba 3 – Pedido con un solo producto

Campo	Descripción
ID	CP-03
Escenario	Creación de cliente con datos incompletos
Requisito Asociado	RF-01 - El sistema debe permitir la creación de clientes válidos
Prioridad	Alta
Escenario Gherkin	Precondición GIVEN el sistema iniciado AND el formulario de creación de cliente disponible Acción WHEN el usuario introduce un nombre válido AND deja el campo teléfono vacío Resultados esperados THEN el sistema detecta datos incompletos AND evita registrar el cliente o muestra advertencia
Estado de Ejecución	Pendiente
Observaciones	Caso negativo para validar entrada de datos.

- La unidad bajo prueba: Datos()

- El caso de prueba: Vamos a probar si creando un nuevo cliente "prueba" se visualizan todos los datos de manera correcta.
- Las aserciones: Resultado esperado: "Nombre: nombre, Apellidos: apellidos, [...]"

- **Arrange (Preparar):** Cliente cliente = new Cliente("Juan", "Perez", "Calle Falsa", "");

- **Act (Actuar)** String resultado = cliente.Datos();

- **Assert (AfirmaryVerificar)** -assertFalse(resultado.contains("Juan"));

Pedido:

En Pedido tenemos más variedad de métodos, para este apartado nos centraremos en "calcularTotal()" para poder implementar diferentes tipos de pruebas.

Caso de prueba 4 – Pedido con un solo producto

Campo	Descripción
ID	CP-04
Escenario	Pedido con un producto
Requisito Asociado	RF-03 - El sistema debe calcular el total del pedido
Prioridad	Media
Escenario Gherkin	Precondición: GIVEN un pedido vacío AND un producto añadido al pedido Acción: WHEN se ejecuta calcularTotal() Resultados esperados: THEN el total del pedido coincide con el precio del producto
Estado de Ejecución	Pendiente
Observaciones	Prueba básica del cálculo.

- La unidad bajo prueba: CalcularTotal()
- El caso de prueba: haremos un pedido nuevo de prueba con un SOLO producto.
- Las aserciones: Verificar que el total del pedido se calcula correctamente según los productos añadidos y sus precios.

- **Arrange (Preparar)**: Producto producto = new Producto("Teclado", 20.0, true); Pedido pedido = new Pedido(); pedido.agregarProducto(producto);

- **Act (Actuar)** double total = pedido.calcularTotal();

- **Assert (AfirmaryVerificar)** - assertEquals(20.0, total);

Caso de prueba 5 – Pedido con varios productos

Campo	Descripción
ID	CP-05
Escenario	Pedido con múltiples productos
Requisito Asociado	RF-03 - Cálculo del total del pedido
Prioridad	Alta
Escenario Gherkin	Precondición: GIVEN un pedido con varios productos añadidos Acción: WHEN se ejecuta calcularTotal() Resultados esperados: THEN el sistema suma correctamente todos los precios
Estado de Ejecución	Pendiente
Observaciones	Válida acumulación de importes.

- La unidad bajo prueba: CalcularTotal()

- El caso de prueba: haremos un pedido nuevo de prueba con VARIOS productos.
- Las aserciones: Verificar que el total del pedido se calcula correctamente según los productos añadidos y sus precios.

- **Arrange (Preparar):** pedido.agregarProducto(new Producto("Ratón", 10.0, true));
pedido.agregarProducto(new Producto("Monitor", 100.0, true));

- **Act (Actuar)** double total = pedido.calcularTotal();

- **Assert (AfirmaryVerificar)** assertEquals(110.0, total);

Caso de prueba 6 – Pedido mixto

Campo	Descripción
ID	CP-06
Escenario	Pedido mixto (producto físico + digital)
Requisito Asociado	RF-03 - El sistema debe calcular correctamente el total del pedido
Prioridad	Alta
Escenario Gherkin	Precondición GIVEN un pedido creado AND existen productos físicos y digitales Acción WHEN se añaden ambos productos al pedido AND se ejecuta el cálculo del total Resultados esperados THEN el sistema suma correctamente los precios finales de cada producto AND devuelve el total combinado correcto
Estado de Ejecución	Pendiente
Observaciones	Valida integración entre clases Producto y Pedido.

- La unidad bajo prueba: CalcularTotal()
- El caso de prueba: haremos un pedido nuevo de prueba con VARIOS productos Mixtos.
- Las aserciones: Verificar que el total del pedido se calcula correctamente según los productos añadidos y sus precios.

- **Arrange (Preparar)**: Producto fisico = new Producto("Libro", 25.0, true);

Producto digital = new Producto("Curso Online", 40.0, false); Pedido pedido = new Pedido();
pedido.agregarProducto(fisico); pedido.agregarProducto(digital);

- **Act (Actuar)** double total = pedido.calcularTotal();

- **Assert (Afirmar/Verificar)** assertNotEquals(110.0, total);

Caso de prueba 7 – Pedido vacío

Campo	Descripción
ID	CP-07
Escenario	Pedido sin productos
Requisito Asociado	RF-03 - Cálculo del total
Prioridad	Media
Escenario Gherkin	Precondición: GIVEN un pedido sin productos Acción: WHEN se ejecuta calcularTotal() Resultados esperados: THEN el total calculado es 0
Estado de Ejecución	Pendiente
Observaciones	Caso límite

- La unidad bajo prueba: CalcularTotal()

- El caso de prueba: haremos un VARIOS pedidos nuevos de prueba.
- Las aserciones: Verificar que el total del pedido se calcula correctamente según los productos añadidos y sus precios.

- **Arrange (Preparar):** Pedido pedido = new Pedido();

- **Act (Actuar)** double total = pedido.calcularTotal();

- **Assert (Afirmar/Verificar)** assertEquals(0.0, total);

Producto:

En Producto solo tenemos el método para calcular el precio del producto dependiendo si es físico o digital así que este será el que utilizaremos para las pruebas.

Caso de prueba 8 – Producto Físico

Campo	Descripción
ID	CP-08
Escenario	Cálculo de precio producto físico
Requisito Asociado	RF-04 - El sistema debe calcular el precio del producto
Prioridad	Alta
Escenario Gherkin	Precondición: GIVEN un producto marcado como físico Acción: WHEN se ejecuta el método de cálculo del precio Resultados esperados: THEN el precio incluye los costes asociados al producto físico
Estado de Ejecución	Pendiente
Observaciones	Comprueba lógica condicional

- La unidad bajo prueba: CalcularPrecio()

- El caso de prueba: comprobaremos que el método funciona con un nuevo producto Físico teniendo en cuenta que debe calcular los gastos de envío.
 - Las aserciones: Verificar que el total del producto se calcula correctamente
- **Arrange (Preparar):** `Producto producto = new Producto("Libro", 30.0, true); // físico`
- **Act (Actuar)** `double precio = producto.calcularPrecio();`
- **Assert (AfirmaryVerificar)** `assertNotEquals(precioNoEsperado, precio);`

Caso de prueba 9 – Producto Digital

Campo	Descripción
ID	CP-09
Escenario	Cálculo de precio producto digital
Requisito Asociado	RF-04 - Cálculo de precio
Prioridad	Alta
Escenario Gherkin	Precondición: GIVEN un producto marcado como digital Acción: WHEN se calcula el precio final Resultados esperados: THEN el precio corresponde únicamente al valor base sin recargos
Estado de Ejecución	Pendiente
Observaciones	Valida diferencia entre tipos de producto.

- La unidad bajo prueba: `CalcularPrecio()`
- El caso de prueba: comprobaremos que el método funciona con un nuevo producto Digital teniendo en cuenta que debe calcular el precio de la licencia .
- Las aserciones: Verificar que el total del producto se calcula correctamente

- **Arrange (Preparar):** `Producto producto = new Producto("Libro", 30.0, true); // digital`
- **Act (Actuar)** `double precio = producto.calcularPrecio();`
- **Assert (AfirmaryVerificar)** `assertNotEquals(precioNoEsperado, precio);`

Caso de prueba 10 – Producto con precio 0

Campo	Descripción
ID	CP-010
Escenario	Producto gratuito
Requisito Asociado	RF-04 - Cálculo de precio
Prioridad	Baja
Escenario Gherkin	Precondición: GIVEN un producto con precio 0 Acción: WHEN se ejecuta el cálculo del precio Resultados esperados: THEN el sistema devuelve precio 0 sin errores
Estado de Ejecución	Pendiente
Observaciones	Caso límite

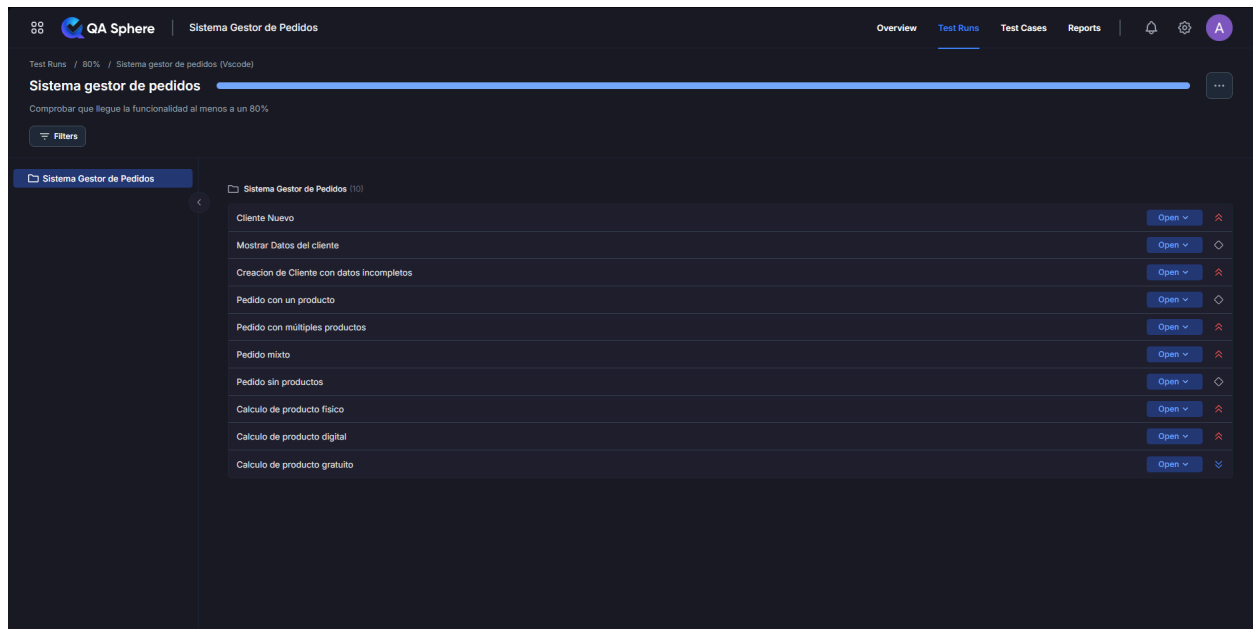
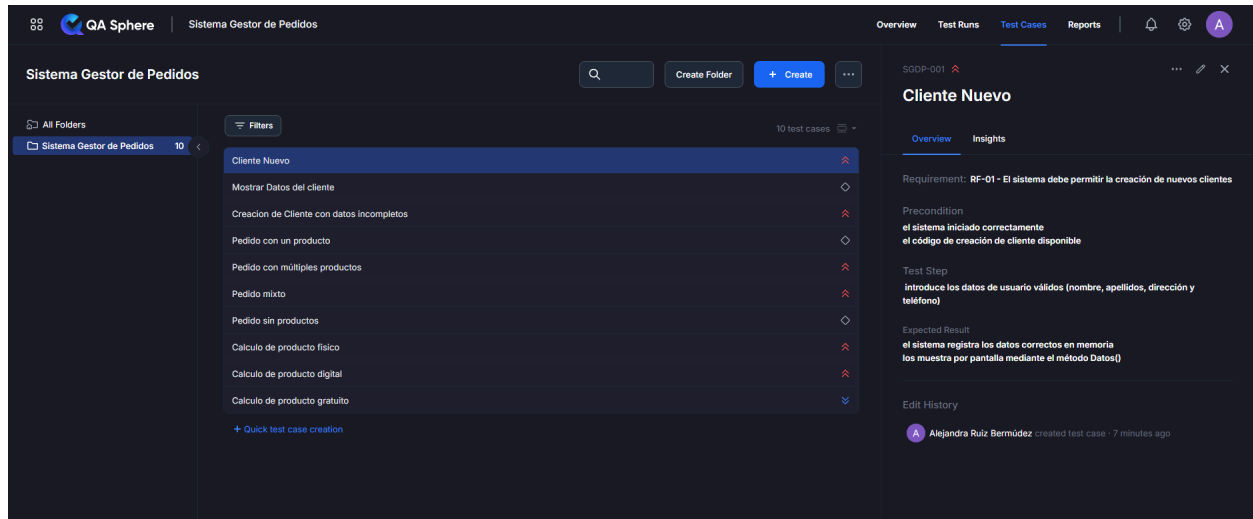
- La unidad bajo prueba: `CalcularPrecio()`
- El caso de prueba: Verificar que el sistema acepta productos gratuitos.
- Las aserciones: Verificar que el total del producto se calcula correctamente

- **Arrange (Preparar):** `Producto producto = new Producto("Demo", 0.0, false);`
- **Act (Actuar)** `double precio = producto.calcularPrecio();`
- **Assert (AfirmaryVerificar)** `assertNotEquals(1.0, precio);`

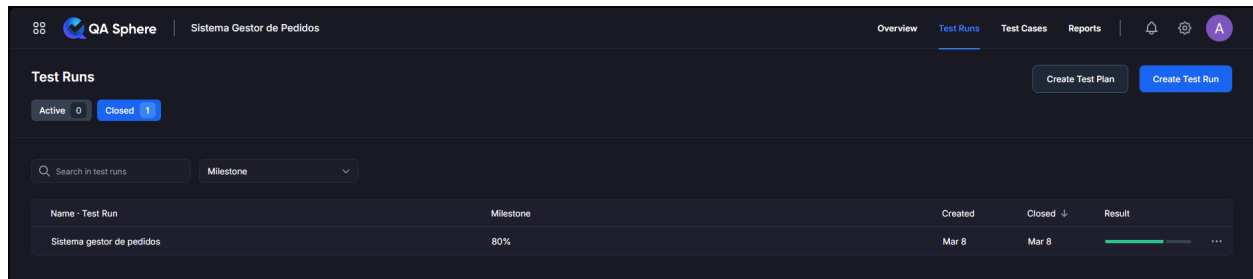
Implementación de Tests

En nuestro código utilizaremos JUnit6 para la comprobación de los casos anteriores (están en la rama /src/tests) y además usaremos la plataforma QASphere para realizar un análisis de calidad.

En la siguiente imagen podemos observar el listado de todos los casos de prueba ya mencionados en el apartado anterior.



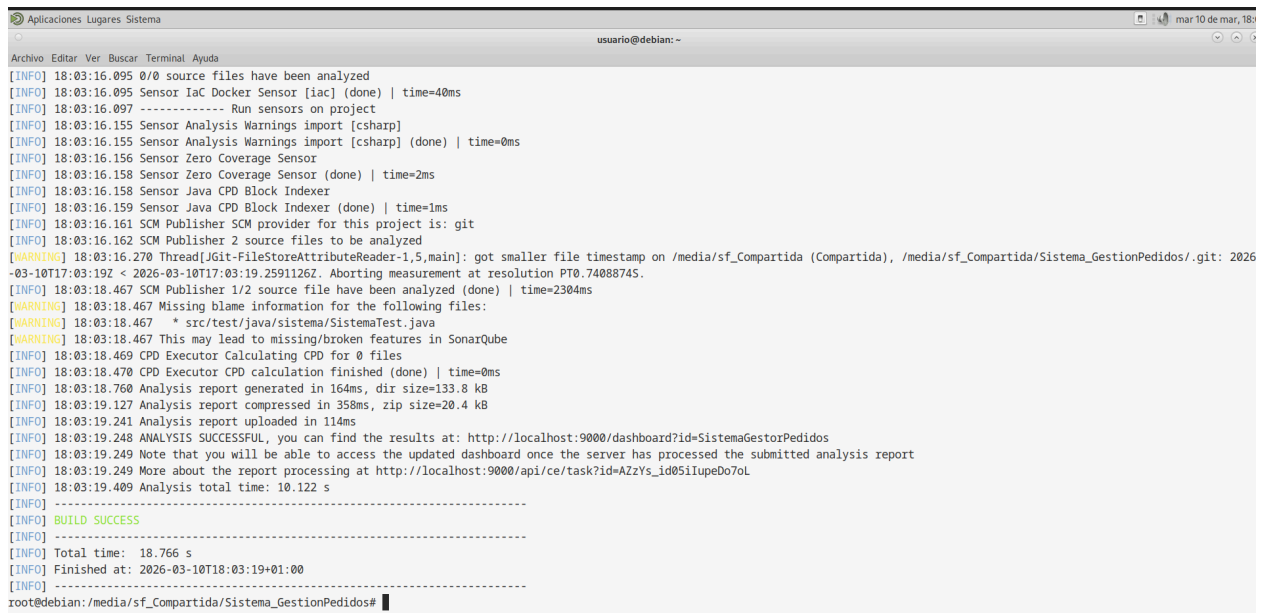
Si funcionan se marcarán como passed, sino como failed y se buscarán los errores. En este caso no hemos tenido ningún problema con los tests realizados así que finalizaríamos tal que así:



SonarQube

En este apartado veremos en el programa SonarQube si llegamos al menos al 80% de funcionamiento. SonarQube es una plataforma de análisis estático diseñada para evaluar y mejorar la calidad, seguridad y mantenibilidad del código fuente. Detecta automáticamente errores (bugs), vulnerabilidades y malas prácticas.

Esto nos agilizará las pruebas.



Automatización, plan de pruebas y testing de la integración.

Para poder implementar los tests automatizados con JUnit 5 / JUnit 6 (siguiendo las directrices modernas de pruebas), debemos considerar primero un factor muy importante que bloquea e invalida la ejecución de tests limpios en SonarQube: el código actual tiene interacciones con consola (`Scanner sc = new Scanner(System.in)`) en métodos de lógica de negocio, como `calcularPrecioFinal()`.

En las pruebas unitarias y de integración automatizadas, no se puede interactuar con el teclado. Por ello, para que los tests funcionen correctamente y no se queden congelados esperando una entrada del usuario, simularemos los escenarios o asumimos que el código será refactorizado para recibir estos parámetros en los métodos.

Clase	Método Crítico	Tipo de Prueba	Descripción / Objetivo
ProductoDigital	<code>aplicarIVA(String tipolva)</code>	Unitario	Validar los cálculos correctos de tasas según el tipo (GENERAL 21%, REDUCIDO 10%, SUPER 4%).
ProductoFisico	<code>calcularCosteEnvio(String pais)</code>	Unitario	Verificar la correcta asignación de costes logísticos según zonas (0€ España, 5€ limítrofes, 10€ resto).
Producto	Constructor / Validación	Gestión de Excepciones	Garantizar que el sistema rechaza la creación de instancias con precios negativos (<code>IllegalArgumentException</code>).

Pedido	calcularTotal()	Parametrizado	Validar la consistencia matemática de la suma total con colecciones dinámicas de productos y cantidades variadas.
Pedido	calcularTotal() (Vacío)	Gestión de Excepciones	Comprobar que no se procesa un contenedor sin ítems (IllegalStateException).
Tienda	realizarVenta(Cliente c, Pedido p)	Integración	Validar la orquestación central: cruce de descuentos de fidelidad (Cliente) con bases imponibles (Pedido).
Factura	Constructor / Desglose	Integración	Asegurar la trazabilidad: generación de códigos autonómicos y el desglose matemático final (Neto, IVA, Envío).

Definición de Pruebas de Sistema

1. Flujo End-to-End(E2E) "Compra completa hasta Facturación", en este caso el objetivo es asegurar que todos los componentes cooperan correctamente en un ciclo de compra real sin intervención de interfaces externas

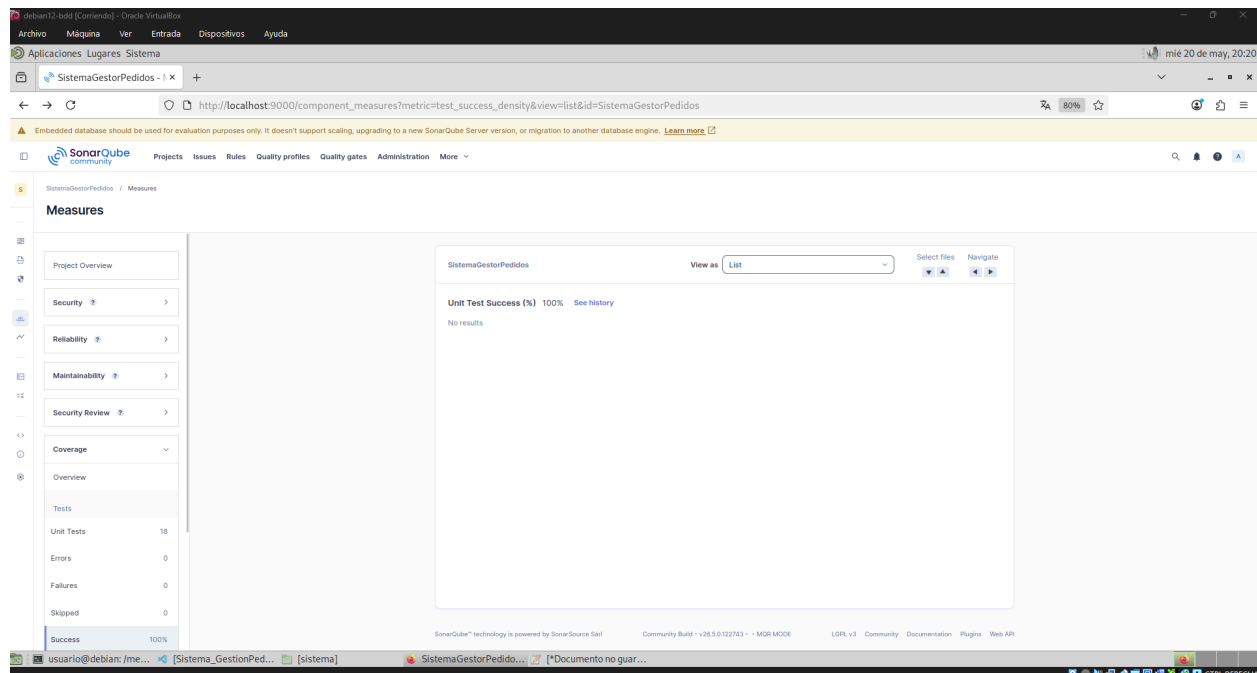
Se recupera el perfil del Cliente (Fidelidad calculada en 15%).

- a) Se instancia un nuevo Pedido asignado al cliente.

- b) Se añaden dinámicamente un ProductoDigital (con IVA General) y un ProductoFisico (destino España, coste 0€).
- c) El pedido calcula el coste acumulado delegando la lógica impositiva en cada producto.
- d) La Tienda procesa la venta, calcula la deducción del 15% sobre el total bruto y genera la Factura.

Resultado Esperado: Factura con código auto-generado (Nombre-DNI-NumPedido), importes netos reducidos por el descuento y persistencia de totales desglosados en consola o archivo.

2. Prueba de Robustez “Manejo de Errores ante Datos inválidos”, en este caso buscamos validar la resiliencia del sistema ante entradas corruptas o acciones fuera de la regla de negocio
 - a) Escenarios Probados:
 - i) Inyección de precios corruptos: Intentar dar de alta un producto con precio -50.0€. El sistema intercepta la anomalía en el constructor y aborta la ejecución de forma segura mediante un `IllegalArgumentException`.
 - ii) Procesamiento fantasma: Intentar llamar al método `calcularTotal()` de un pedido recién creado al que todavía no se le han añadido productos. El sistema bloquea el flujo lanzando un `IllegalStateException`.
 - b) Resultado Esperado: Ningún error interrumpe la aplicación (no hay crashes), las excepciones son controladas y no se permite avanzar hacia la fase de facturación con datos inconsistentes.
3. Pruebas de Regresión (Comentarios del Proyecto)
 - a) Nota de Diseño: Todos los casos previos de pruebas unitarias estándar se mantienen operativos en la suite (`src/test/java`). Cualquier refactorización del código orientada a limpiar los bloques de lectura interactiva por consola (Scanner) debe validar que los resultados históricos de impuestos y costes de envío sigan devolviendo un valor idéntico para asegurar que los cambios nuevos no rompan la lógica vieja.



(La capturita de SonnarQube con un 100% de coverage)

Conclusión

Como conclusión, el desarrollo de este proyecto ha permitido recorrer de forma completa el proceso de diseño y construcción de un sistema orientado a objetos, comenzando por la elaboración del diagrama UML y finalizando con la implementación en Java. A través de este flujo de trabajo, se ha demostrado la importancia de planificar adecuadamente la estructura del programa antes de escribir código, ya que el modelo conceptual facilitó la organización de las clases, sus atributos, sus métodos y las relaciones entre ellas.

La comparación entre el diagrama inicial y el código generado por ingeniería inversa confirma que el diseño planteado fue coherente y correctamente aplicado, puesto que ambos diagramas presentan una estructura prácticamente equivalente.

Aunque el código funciona correctamente y refleja bien el diseño inicial, existen diversas mejoras que se podrían implementar:

- Agregar una validación de datos, se podrían sanear las respuestas introducidas por el usuario para verificar si son válidas o no.
- Implementar manejo de excepciones, especialmente en las operaciones como los cálculos, creación de objetos o gestión de listas.



En cuanto al apartado de QA de pruebas ha sido interesante poner a prueba, nunca mejor dicho, el programa, no ha habido bugs notables en las partes más básicas ni tampoco errores.

Lo único que me dió problemas fueron los tokens del SonarQube, en mi caso estoy utilizando un Debian 12, cuya solución fue pasarle el token por variable del sistema porque por alguna razón no está autorizada si la coge del pom.xml.

Bibliografía

Todo el proyecto se irá registrando en Github:

https://github.com/Alerb25/Sistema_GestionPedidos